



Weaviate

Data ingestion strategies for **Vector** Databases



JP Hwang

Technical curriculum developer

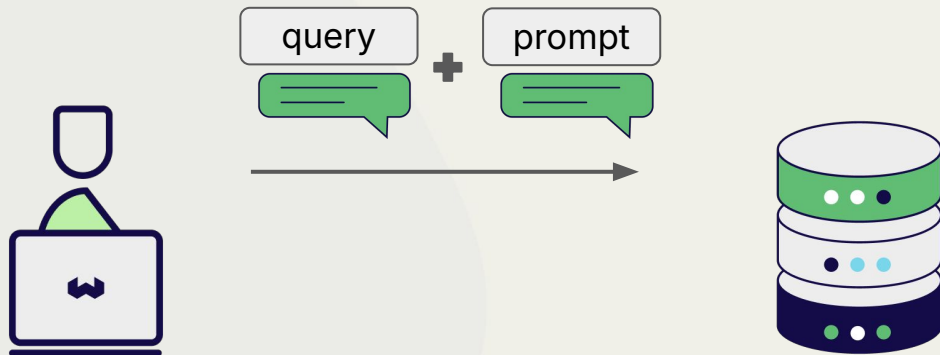


Why use a **vector** database?

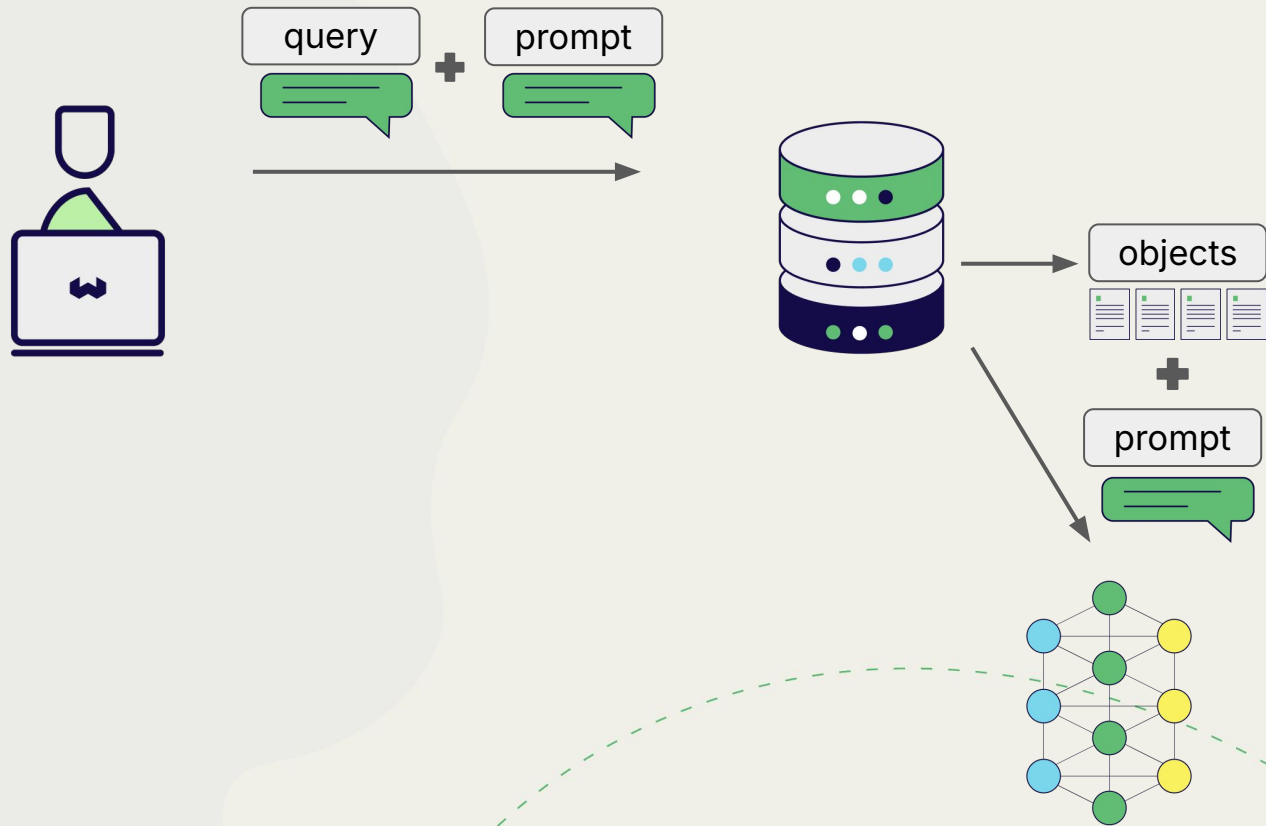
Most commonly:

Retrieval augmented generation (**RAG**)

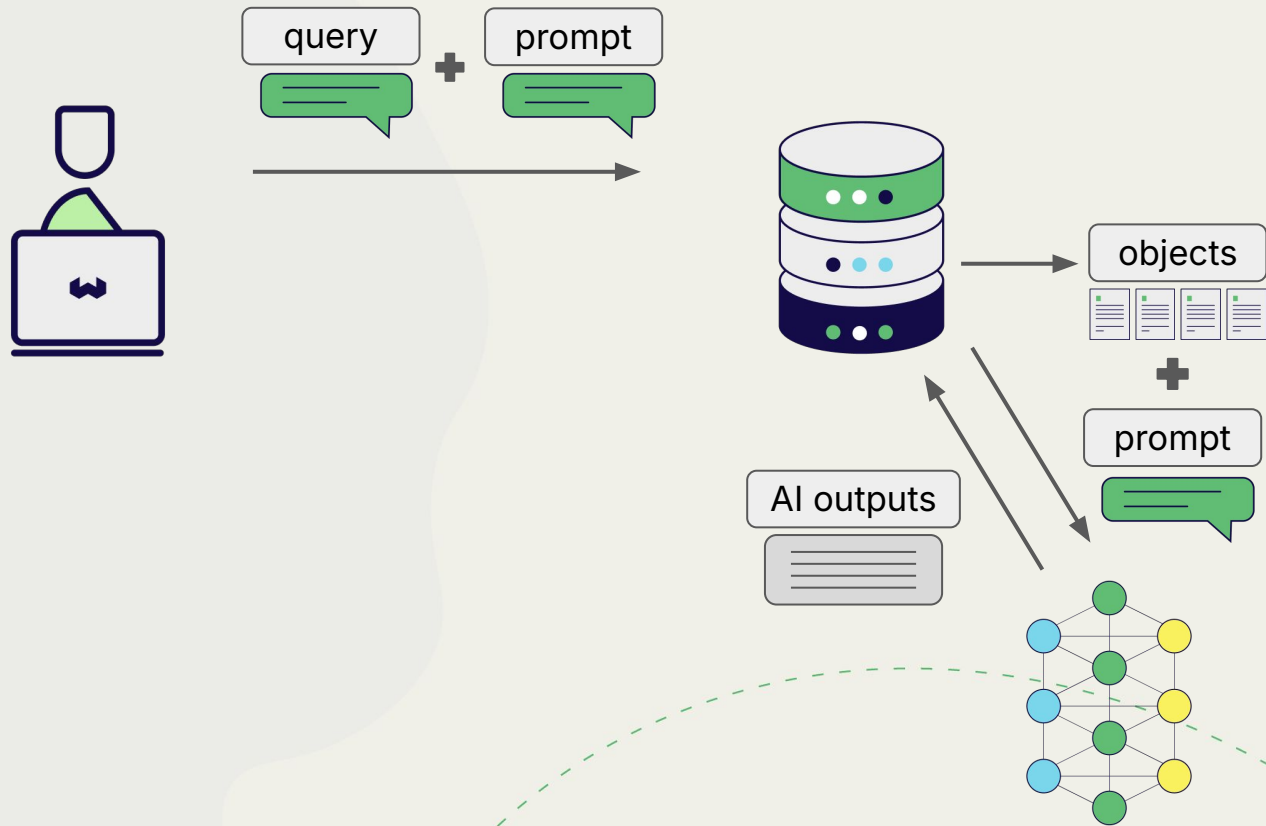
RAG



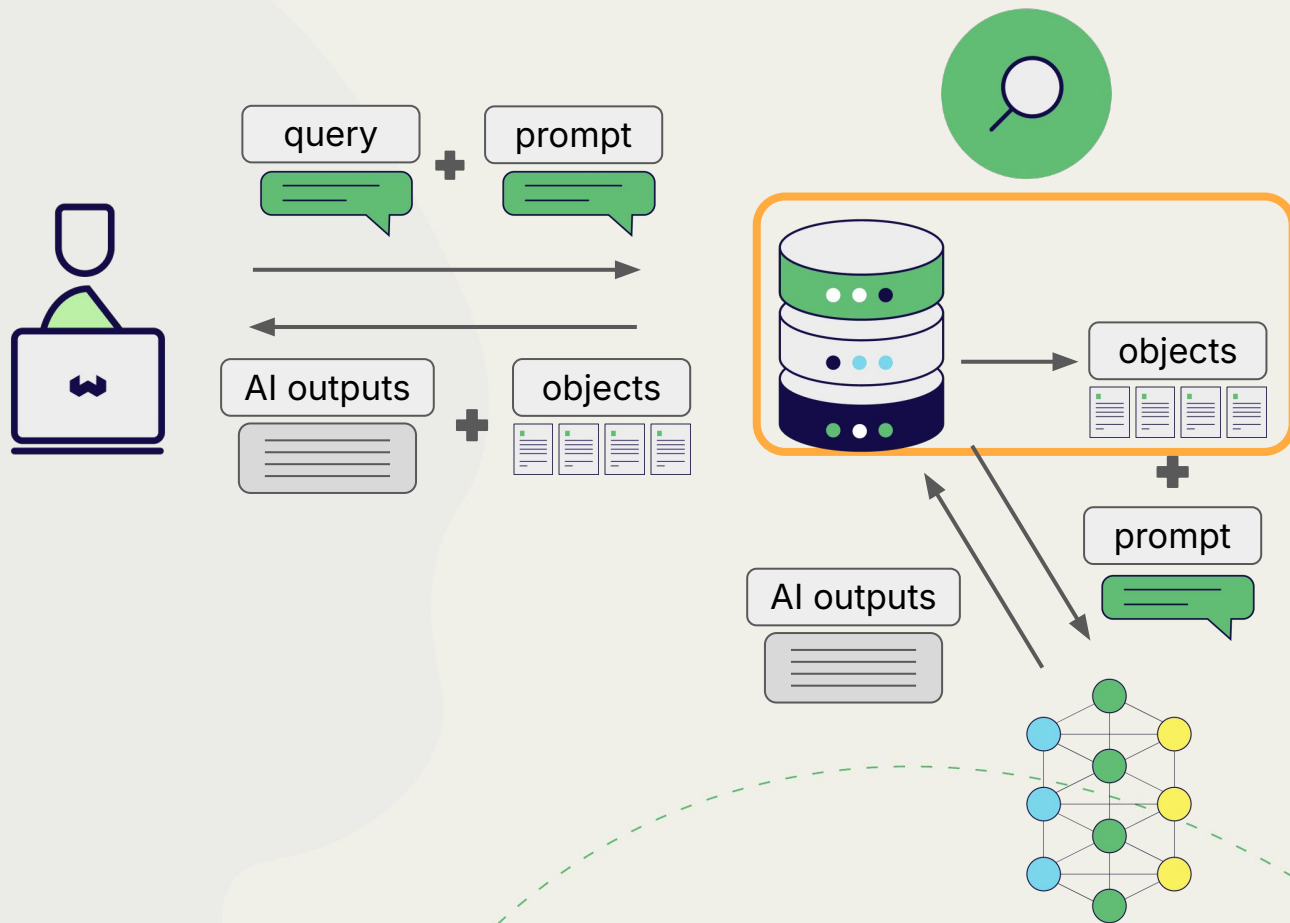
RAG



RAG



RAG





Key topics

Vectorizer model choice

Find the right translator

Chunking

Start with end in mind

Quantization

Resource management

Vector construction

Define “similarity”



Key topics

Vectorizer model choice

Find the right translator

Chunking

Start with end in mind

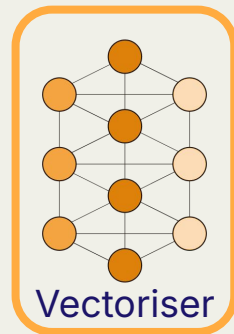
Quantization

Resource management

Vector construction

Define “similarity”

Vectoriser: input ➡ vectors



Hundreds of models are available:

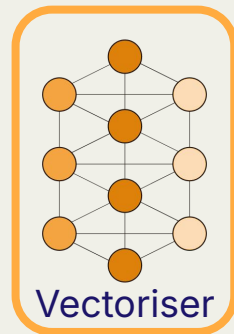
- Proprietary models
 - Via Cohere, OpenAI, Google, AWS, etc.
- Open-source models
 - Via Hugging Face
- Can be overwhelming

Vectoriser: input ➡ vectors

Some significant models

- Word2Vec (2013)
- GloVe (Global Vectors for Word Representation) (2014)
- FastText (2016)
- ELMo (Embeddings from Language Models) (2018)
- BERT (Bidirectional Encoder Representations from Transformers) (2018)
- RoBERTa (Robustly Optimized BERT Pretraining Approach) (2019)
- DistilBERT (2019)
- T5 (Text-To-Text Transfer Transformer) (2019)
- CLIP (Contrastive Language-Image Pretraining) (2020)
- DeBERTa (Decoding-enhanced BERT with Disentangled Attention) (2020)
- Sentence-BERT (SBERT) (2020)
- Ada-002 (2021)
- Embed-multilingual-v2.0 (2022)
- ImageBind (2023)

Vectoriser choice



Choose a baseline model

- Do you have access to a popular model?
 - Cohere **v3**; OpenAI **embedding-3**; Snowflake **arctic**
- Is there a specific need?
 - Domain (legal, medical, coding)
 - Size / inference speed / cost
- How to get started?



Massive Text Embedding Benchmark (MTEB) Leaderboard. To submit, refer to the [MTEB GitHub repository](#) 🤖 Refer to the [MTEB paper](#) for details on metrics, tasks and models.

Search Bar (separate multiple queries with `;`)

🔍 Search for a model and press enter...

Model types

☒ Open

☒ Proprietary

☒ Sentence Transformers

☒ Cross-Encoders

☒ Bi-Encoders

Model sizes (in number of parameters)

☒ <100M

☒ 100M to 250M

☒ 250M to 500M

☒ 500M to 1B

☒ >1B

Overall

Bitext Mining

Classification

Clustering

Pair Classification

Reranking

Retrieval

STS

Summarization

Retrieval w/Instructions

English

Chinese

French

Polish

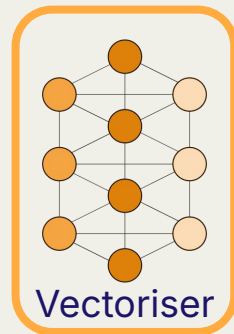
Overall MTEB English leaderboard 🏆

◦ Metric: Various, refer to task tabs

◦ Languages: English

Rank	Model	Model Size (Million Parameters)	Memory Usage (GB, fp32)	Embedding Dimensions	Max Tokens	Average (56 datasets)	C A d
1	NV-Embed-v1	7851	29.25	4096	32768	69.32	8
2	voyage-large-2-instruct			1024	16000	68.28	8
3	Linq-Embed-Mistral	7111	26.49	4096	32768	68.17	8
4	SFR-Embedding-Mistral	7111	26.49	4096	32768	67.56	7
5	gte-Qwen1.5-7B-instruct	7099	26.45	4096	32768	67.34	7

Vectoriser choice



Evaluate & iterate

- Benchmarks are good but...
 - Evaluate on your own use case
 - Create a small validation dataset
 - Trust "gut feel"
 - You are the expert!



Key topics

Vectorizer model choice

Horses for courses

Chunking

Start with end in mind

Quantization

Resource management

Vector construction

Define “similarity”



Chunking

Mixture of experts [\[edit \]](#)

Main article: *Mixture of experts*

The largest LLM may be too expensive to train and use directly. For such models, *mixture of experts* (MoE) can be applied, a line of research pursued by Google researchers since 2017 to train models reaching up to 1 trillion parameters.^[REDACTED]

Prompt engineering, attention mechanism, and context window [\[edit \]](#)

See also: *Prompt engineering* and *Attention (machine learning)*

Most results previously achievable only by (costly) fine-tuning, can be achieved through *prompt engineering*, although limited to the scope of a single conversation (more precisely, limited to the scope of a context window).^[1]

In order to find out which tokens are relevant to each other within the scope of the context window, the attention mechanism calculates "soft" weights for each token, more precisely for its embedding, by using multiple attention heads, each with its own "relevance" for calculating its own soft weights. For example, the small (i.e. 117M parameter sized) GPT-2 model, has had twelve attention heads and a context window of only 1x token.^[2] In its medium version it has 343M parameters and contains 24 layers, each with 12 attention heads. For the training with gradient descent a batch size of 512 was utilized.^[3]

The largest models, such as Google's *Gemini 1.5*, presented in February 2024, can have a context window sized up to 1 million (context window of 10 million was also "successfully tested").^[4] Other models with large context windows includes Anthropic's Claude 2.1, with a context window of up to 200k tokens.^[5] Note that this maximum refers to the number of input tokens and that the maximum number of output tokens differs from the input and is often smaller. For example, the GPT-4 Turbo model has a maximum output of 4096 tokens.^[6]

Length of a conversation that the model can take into account when generating its next answer is limited by the size of a context window, as well. If the length of a conversation, for example with *Chat-GPT*, is longer than its context window, only the parts inside the context window are taken into account when generating the next answer, or the model needs to apply some algorithm to summarize the too distant parts of conversation.

The shortcomings of making a context window larger include higher computational cost and possibly diluting the focus on local context, while making it smaller can cause a model to miss an important long-range dependency. Balancing them are a matter of experimental and domain-specific considerations.

A model may be pre-trained either to predict how the segment continues, or what is missing in the segment, given a segment from its training dataset.^[7] It can be either

- **autoregressive** (i.e. predicting how the segment continues, the way GPTs do it): for example given a segment "I like to eat", the model predicts "ice cream", or "sushi".
- **masking** (i.e. filling in the parts missing from the segment, the way BERT^[8] does it): for example, given a segment "I like to [] [] cream", the model predicts that "eat" and "ice" are missing.

Models may be trained on auxiliary tasks which test their understanding of the data distribution, such as Next Sentence Prediction (NSP), in which pairs of sentences are presented and the model must predict whether they appear consecutively in the training corpus.^[9] During training, *regularization* loss is also used to stabilize training. However regularization loss is usually not used during *testing* and *evaluation*.

Training cost [\[edit \]](#)

Advances in software and hardware have reduced the cost substantially since 2020, such that in 2023 training of a 12-billion-parameter LLM computational cost is 75,300 A100-GPU-hours, while in 2020 the cost of training a 1.5-billion-parameter LLM (which was two orders of magnitude smaller than the state of the art in 2020) was between \$80 thousand and \$1.4 million.^{[10][11]} Since 2020, large sums were invested in increasingly large models. For example, training of the GPT-2 (i.e. a 1.5-billion-parameters model) in 2019 cost \$50,000, while training of the PaLM (i.e. a 540-billion-parameters model) in 2022 cost \$8 million, and Megatron-Turing NLG G30B (in 2023) cost around \$11 million.^[12]

For Transformer-based LLM, training cost is much higher than inference cost. It costs 6 FLOP/h per parameter to train on one token, whereas it costs 1 to 2 FLOP/h per parameter to infer on one token.^[13]

Mixture of experts [\[edit \]](#)

Main article: *Mixture of experts*

The original LLM was too expensive to train and use directly. For such models, *mixture of experts* (MoE) can be applied, a line of research pursued by Google researchers since 2017 to train models reaching up to 1 trillion parameters.^[REDACTED]

Prompt engineering, attention mechanism, and context window [\[edit \]](#)

See also: *Prompt engineering* and *Attention (machine learning)*

Most results previously achievable only by (costly) fine-tuning, can be achieved through *prompt engineering*, although limited to the scope of a single conversation (more precisely, limited to the scope of a context window).^[1]

In order to find out which tokens are relevant to each other within the scope of the context window, the attention mechanism calculates "soft" weights for each token, more precisely for its embedding, by using multiple attention heads, each with its own "relevance" for calculating its own soft weights. For example, the small (i.e. 117M parameter sized) GPT-2 model, has had twelve attention heads and a context window of only 1x token.^[2] In its medium version it has 343M parameters and contains 24 layers, each with 12 attention heads. For the training with gradient descent a batch size of 512 was utilized.^[3]

The largest models, such as Google's *Gemini 1.5*, presented in February 2024, can have a context window sized up to 1 million (context window of 10 million was also "successfully tested").^[4] Other models with large context windows includes Anthropic's Claude 2.1, with a context window of up to 200k tokens.^[5] Note that this maximum refers to the number of input tokens and that the maximum number of output tokens differs from the input and is often smaller. For example, the GPT-4 Turbo model has a maximum output of 4096 tokens.^[6]

Length of a conversation that the model can take into account when generating its next answer is limited by the size of a context window, as well. If the length of a conversation, for example with *Chat-GPT*, is longer than its context window, only the parts inside the context window are taken into account when generating the next answer, or the model needs to apply some algorithm to summarize the too distant parts of conversation.

The shortcomings of making a context window larger include higher computational cost and possibly diluting the focus on local context, while making it smaller can cause a model to miss an important long-range dependency. Balancing them are a matter of experimentation and domain-specific considerations.

A model may be pre-trained either to predict how the segment continues, or what is missing in the segment, given a segment from its training dataset.^[7] It can be either

- **autoregressive** (i.e. predicting how the segment continues, the way GPTs do it): for example given a segment "I like to eat", the model predicts "ice cream", or "sushi".
- **masking** (i.e. filling in the parts missing from the segment, the way BERT^[8] does it): for example, given a segment "I like to [] [] cream", the model predicts that "eat" and "ice" are missing.

Models may be trained on auxiliary tasks which test their understanding of the data distribution, such as Next Sentence Prediction (NSP), in which pairs of sentences are presented and the model must predict whether they appear consecutively in the training corpus.^[9] During training, *regularization* loss is also used to stabilize training. However regularization loss is usually not used during *testing* and *evaluation*.

Training cost [\[edit \]](#)

Advances in software and hardware have reduced the cost substantially since 2020, such that in 2023 training of a 12-billion-parameter LLM computational cost is 75,300 A100-GPU-hours, while in 2020 the cost of training a 1.5-billion-parameter LLM (which was two orders of magnitude smaller than the state of the art in 2020) was between \$80 thousand and \$1.4 million.^{[10][11]} Since 2020, large sums were invested in increasingly large models. For example, training of the GPT-2 (i.e. a 1.5-billion-parameters model) in 2019 cost \$50,000, while training of the PaLM (i.e. a 540-billion-parameters model) in 2022 cost \$8 million, and Megatron-Turing NLG G30B (in 2023) cost around \$11 million.^[12]

For Transformer-based LLM, training cost is much higher than inference cost. It costs 6 FLOP/h per parameter to train on one token, whereas it costs 1 to 2 FLOP/h per parameter to infer on one token.^[13]



The diagram illustrates the attention mechanism. It shows a sequence of tokens (The, dog, was, barking, loudly, in, the, yard) and their corresponding embeddings. Arrows represent the attention weights calculated between the query embedding (for 'The') and the key-value pairs of other tokens. The diagram shows that 'The' has high attention weights for 'dog' and 'was', indicating they are most relevant to its context.



Chunking

Mixture of experts [\[edit\]](#)

Main article: *Mixture of experts*

The target LLM may be too expensive to train and use directly. For such models, *mixture of experts* (MoE) can be applied, a line of research pursued by Google researchers since 2017 to train models reaching up to 1 trillion parameters.^[REDACTED]

Prompt engineering, attention mechanism, and context window [\[edit\]](#)

See also: *Prompt engineering* and *Attention (machine learning)*

Most results previously achievable only by (costly) fine-tuning, can be achieved through *prompt engineering*, although limited to the scope of a single conversation (more precisely, limited to the scope of context window)^[3]

In order to find out which tokens are relevant to each other within the scope of the context window, the attention mechanism calculates "soft" weights for each token, more precisely for its embedding, by using multiple attention heads, each with its own "relevance" for calculating its own soft weights. For example, the small (i.e. 117M parameter sized) GPT-2 model, has had twelve attention heads and a context window of only 1x token^[20]. In its medium version it has 343M parameters and contains 24 layers, each with 12 attention heads. For the training with gradient descent a batch size of 512 was utilized.^[3]

The largest models, such as Google's *Gemini 1.5*, presented in February 2024, can have a context window sized up to 1 million (context window of 10 million was also "successfully tested").^[16] Other models with large context windows includes Anthropic's Claude 2.1, with a context window of up to 200k tokens.^[21] Note that this context refers to the number of input tokens and not the maximum number of output tokens differs from the input and is often smaller. For example, the GPT-4 Turbo model has a maximum output of 4096 tokens.^[26]

Length of a conversation that the model can take into account when generating its next answer is limited by the size of a context window, as well. If the length of a conversation, for example with *Chat-GPT*, is longer than its context window, only the parts inside the context window are taken into account when generating the next answer, or the model needs to apply some algorithm to summarize the too distant parts of conversation.

The shortcomings of making a context window larger include higher computational cost and possibly diluting the focus on local context, while making it smaller can cause a model to miss an important long-range dependency. Balancing them are a matter of experimental and domain-specific considerations.

A model may be pre-trained either to predict how the segment continues, or what is missing in the segment, given a segment from its training dataset.^[27] It can be either

- **autoregressive** (i.e. predicting how the segment continues, the way GPTs do it): for example given a segment "I like to eat", the model predicts "ice cream", or "honey".
- **maskLM** (i.e. filling in the parts missing from the segment, the way BERT^[28] does it): for example, given a segment "I like to [] [] cream", the model predicts that "eat" and "ice" are missing.

Models may be trained on auxiliary tasks which test their understanding of the data distribution, such as Next Sentence Prediction (NSP), in which pairs of sentences are presented and the model must predict whether they appear consecutively in the training corpus.^[29] During training, *regularization* loss is also used to stabilize training. However regularization loss is usually not used during *testing* and evaluation.

Training cost [\[edit\]](#)

Advances in software and hardware have reduced the cost substantially since 2020, such that in 2023 training of a 12-billion-parameter LLM computational cost is 73,200 A100-GPU-hours, while in 2020 the cost of training a 1.5-billion-parameter LLM which was two orders of magnitude smaller than the state of the art in 2020 was between \$80 thousand and \$1.6 million.^{[30][31]} Since 2020, large sums were invested in increasingly large models. For example, training of the GPT-2 (i.e. a 1.5-billion-parameters model) in 2019 cost \$50,000, while training of the PaLM (i.e. a 540-billion-parameters model) in 2022 cost \$8 million, and Megatron-Turing NLG G30B (in 2022) cost around \$11 million.^[42]

For Transformer-based LLM, training cost is much higher than inference cost. It costs 6 FLOPs per parameter to train on one token, whereas it costs 1 to 2 FLOPs per parameter to infer on one token.^[43]

Mixture of experts [\[edit\]](#)

Main article: *Mixture of experts*

The target LLM may be too expensive to train and use directly. For such models, *mixture of experts* (MoE) can be applied, a line of research pursued by Google researchers since 2017 to train models reaching up to 1 trillion parameters.^[REDACTED]

Prompt engineering, attention mechanism, and context window [\[edit\]](#)

See also: *Prompt engineering* and *Attention (machine learning)*

Most results previously achievable only by (costly) fine-tuning, can be achieved through *prompt engineering*, although limited to the scope of a single conversation (more precisely, limited to the scope of context window)^[3]

In order to find out which tokens are relevant to each other within the scope of the context window, the attention mechanism calculates "soft" weights for each token, more precisely for its embedding, by using multiple attention heads, each with its own "relevance" for calculating its own soft weights. For example, the small (i.e. 117M parameter sized) GPT-2 model, has had twelve attention heads and a context window of only 1x token^[20]. In its medium version it has 343M parameters and contains 24 layers, each with 12 attention heads. For the training with gradient descent a batch size of 512 was utilized.^[3]

The largest models, such as Google's *Gemini 1.5*, presented in February 2024, can have a context window sized up to 1 million (context window of 10 million was also "successfully tested").^[16] Other models with large context windows includes Anthropic's Claude 2.1, with a context window of up to 200k tokens.^[21] Note that this context refers to the number of input tokens and not the maximum number of output tokens differs from the input and is often smaller. For example, the GPT-4 Turbo model has a maximum output of 4096 tokens.^[26]

Length of a conversation that the model can take into account when generating its next answer is limited by the size of a context window, as well. If the length of a conversation, for example with *Chat-GPT*, is longer than its context window, only the parts inside the context window are taken into account when generating the next answer, or the model needs to apply some algorithm to summarize the too distant parts of conversation.

The shortcomings of making a context window larger include higher computational cost and possibly diluting the focus on local context, while making it smaller can cause a model to miss an important long-range dependency. Balancing them are a matter of experimental and domain-specific considerations.

A model may be pre-trained either to predict how the segment continues, or what is missing in the segment, given a segment from its training dataset.^[27] It can be either

- **autoregressive** (i.e. predicting how the segment continues, the way GPTs do it): for example given a segment "I like to eat", the model predicts "ice cream", or "honey".
- **maskLM** (i.e. filling in the parts missing from the segment, the way BERT^[28] does it): for example, given a segment "I like to [] [] cream", the model predicts that "eat" and "ice" are missing.

Models may be trained on auxiliary tasks which test their understanding of the data distribution, such as Next Sentence Prediction (NSP), in which pairs of sentences are presented and the model must predict whether they appear consecutively in the training corpus.^[29] During training, *regularization* loss is also used to stabilize training. However regularization loss is usually not used during *testing* and evaluation.

Training cost [\[edit\]](#)

Advances in software and hardware have reduced the cost substantially since 2020, such that in 2023 training of a 12-billion-parameter LLM computational cost is 73,200 A100-GPU-hours, while in 2020 the cost of training a 1.5-billion-parameter LLM which was two orders of magnitude smaller than the state of the art in 2020 was between \$80 thousand and \$1.6 million.^{[30][31]} Since 2020, large sums were invested in increasingly large models. For example, training of the GPT-2 (i.e. a 1.5-billion-parameters model) in 2019 cost \$50,000, while training of the PaLM (i.e. a 540-billion-parameters model) in 2022 cost \$8 million, and Megatron-Turing NLG G30B (in 2022) cost around \$11 million.^[42]

For Transformer-based LLM, training cost is much higher than inference cost. It costs 6 FLOPs per parameter to train on one token, whereas it costs 1 to 2 FLOPs per parameter to infer on one token.^[43]

Mixture of experts [\[edit\]](#)

Main article: *Mixture of experts*

The target LLM may be too expensive to train and use directly. For such models, *mixture of experts* (MoE) can be applied, a line of research pursued by Google researchers since 2017 to train models reaching up to 1 trillion parameters.^[REDACTED]

Prompt engineering, attention mechanism, and context window [\[edit\]](#)

See also: *Prompt engineering* and *Attention (machine learning)*

Most results previously achievable only by (costly) fine-tuning, can be achieved through *prompt engineering*, although limited to the scope of a single conversation (more precisely, limited to the scope of context window)^[3]

In order to find out which tokens are relevant to each other within the scope of the context window, the attention mechanism calculates "soft" weights for each token, more precisely for its embedding, by using multiple attention heads, each with its own "relevance" for calculating its own soft weights. For example, the small (i.e. 117M parameter sized) GPT-2 model, has had twelve attention heads and a context window of only 1x token^[20]. In its medium version it has 343M parameters and contains 24 layers, each with 12 attention heads. For the training with gradient descent a batch size of 512 was utilized.^[3]

The largest models, such as Google's *Gemini 1.5*, presented in February 2024, can have a context window sized up to 1 million (context window of 10 million was also "successfully tested").^[16] Other models with large context windows includes Anthropic's Claude 2.1, with a context window of up to 200k tokens.^[21] Note that this context refers to the number of input tokens and not the maximum number of output tokens differs from the input and is often smaller. For example, the GPT-4 Turbo model has a maximum output of 4096 tokens.^[26]

Length of a conversation that the model can take into account when generating its next answer is limited by the size of a context window, as well. If the length of a conversation, for example with *Chat-GPT*, is longer than its context window, only the parts inside the context window are taken into account when generating the next answer, or the model needs to apply some algorithm to summarize the too distant parts of conversation.

The shortcomings of making a context window larger include higher computational cost and possibly diluting the focus on local context, while making it smaller can cause a model to miss an important long-range dependency. Balancing them are a matter of experimental and domain-specific considerations.

A model may be pre-trained either to predict how the segment continues, or what is missing in the segment, given a segment from its training dataset.^[27] It can be either

- **autoregressive** (i.e. predicting how the segment continues, the way GPTs do it): for example given a segment "I like to eat", the model predicts "ice cream", or "honey".
- **maskLM** (i.e. filling in the parts missing from the segment, the way BERT^[28] does it): for example, given a segment "I like to [] [] cream", the model predicts that "eat" and "ice" are missing.

Models may be trained on auxiliary tasks which test their understanding of the data distribution, such as Next Sentence Prediction (NSP), in which pairs of sentences are presented and the model must predict whether they appear consecutively in the training corpus.^[29] During training, *regularization* loss is also used to stabilize training. However regularization loss is usually not used during *testing* and evaluation.

Training cost [\[edit\]](#)

Advances in software and hardware have reduced the cost substantially since 2020, such that in 2023 training of a 12-billion-parameter LLM computational cost is 73,200 A100-GPU-hours, while in 2020 the cost of training a 1.5-billion-parameter LLM which was two orders of magnitude smaller than the state of the art in 2020 was between \$80 thousand and \$1.6 million.^{[30][31]} Since 2020, large sums were invested in increasingly large models. For example, training of the GPT-2 (i.e. a 1.5-billion-parameters model) in 2019 cost \$50,000, while training of the PaLM (i.e. a 540-billion-parameters model) in 2022 cost \$8 million, and Megatron-Turing NLG G30B (in 2022) cost around \$11 million.^[42]

For Transformer-based LLM, training cost is much higher than inference cost. It costs 6 FLOPs per parameter to train on one token, whereas it costs 1 to 2 FLOPs per parameter to infer on one token.^[43]



Chunking

Mixture of experts [\[edit\]](#)

Main article: *Mixture of experts*

The largest LLM may be too expensive to train and use directly. For such models, **mixture of experts** (MoE) can be applied, a line of research pursued by Google researchers since 2017 to train models reaching up to 1 trillion parameters.^[REDACTED]

Prompt engineering, attention mechanism, and context window [\[edit\]](#)

See also: *Prompt engineering* and *Attention (machine learning)*

Most results previously achievable only by (costly) fine-tuning, can be achieved through **prompt engineering**, although limited to the scope of a single conversation (more precisely, limited to the scope of a context window)^[REDACTED]

In order to find out which tokens are relevant to each other within the scope of the context window, the attention mechanism calculates "soft" weights for each token, more precisely for its embedding, by using multiple attention heads, each with its own "relevance" for calculating its own soft weights. For example, the small (i.e. 117M parameter sized) GPT-2 model, has had twelve attention heads and a context window of only 1x token^[REDACTED] in its medium version it has 343M parameters and contains 24 layers, each with 12 attention heads. For the training with gradient descent a batch size of 512 was utilized.^[REDACTED]



The largest models, such as Google's *Gemini 1.5*, presented in February 2024, can have a context window sized up to 1 million (context window of 10 million was also "successfully tested").^[REDACTED] Other models with large context windows includes Anthropic's Claude 2.1, with a context window of up to 200k tokens.^[REDACTED] Note that this maximum refers to the number of input tokens and that the maximum number of output tokens differs from the input and is often smaller. For example, the GPT-4 Turbo model has a maximum output of 4096 tokens.^[REDACTED]

Length of a conversation that the model can take into account when generating its next answer is limited by the size of a context window, as well. If the length of a conversation, for example with *Chat-GPT*, is longer than its context window, only the parts inside the context window are taken into account when generating the next answer, or the model needs to apply some algorithm to summarize the too distant parts of conversation.

The shortcomings of making a context window larger include higher computational cost and possibly diluting the focus on local context, while making it smaller can cause a model to miss an important long-range dependency. Balancing them are a matter of experimental and domain-specific considerations.

A model may be pre-trained either to predict how the segment continues, or what is missing in the segment, given a segment from its training dataset.^[REDACTED] It can be either

- **autoregressive** (i.e. predicting how the segment continues, the way GPTs do it): for example given a segment "I like to eat", the model predicts "ice cream", or "honey".
- **masking** (i.e. filling in the parts missing from the segment, the way BERT^[REDACTED] does it): for example, given a segment "I like to eat [] [] cream", the model predicts that "eat" and "ice" are missing.

Models may be trained on auxiliary tasks which test their understanding of the data distribution, such as Next Sentence Prediction (NSP), in which pairs of sentences are presented and the model must predict whether they appear consecutively in the training corpus.^[REDACTED] During training, **regularization** loss is also used to stabilize training. However regularization loss is usually not used during **testing** and evaluation.

Training cost [\[edit\]](#)

Advances in software and hardware have reduced the cost substantially since 2020, such that in 2023 training of a 12-billion-parameter LLM computational cost is 7.20k A100-GPU-hours, while in 2020 the cost of training a 1.5-billion-parameter LLM was two orders of magnitude smaller than the state of the art in 2020 was between \$80 and \$140 million.^[REDACTED] Since 2020, large sums were invested in increasingly large models. For example, training of the GPT-2 (i.e. a 1.5-billion-parameters model) in 2019 cost \$50,000, while training of the PaLM (i.e. a 540-billion-parameters model) in 2022 cost \$8 million, and Megatron-Turing NLO G30B (in 2022) cost around \$11 million.^[REDACTED]

For Transformer-based LLM, training cost is much higher than inference cost. It costs 6 FLOPs per parameter to train on one token, whereas it costs 1 to 2 FLOPs per parameter to infer on one token.^[REDACTED]

MOUSE OF EXPERTS [\[edit\]](#)

MoE article: *Mixture of experts*

The original LLM was too expensive to train and use directly. For such models, **mixture of experts** (MoE) can be applied, a line of research pursued by Google researchers since 2017 to train models reaching up to 1 trillion parameters.^[REDACTED]

Prompt engineering, attention mechanism, and context window [\[edit\]](#)

See also: *Prompt engineering* and *Attention (machine learning)*

Most results previously achievable only by (costly) fine-tuning, can be achieved through **prompt engineering**, although limited to the scope of a single conversation (more precisely, limited to the scope of a context window)^[REDACTED]

Length of a conversation that the model can take into account when generating its next answer is limited by the size of a context window, as well. If the length of a conversation, for example with *Chat-GPT*, is longer than its context window, only the parts inside the context window are taken into account when generating the next answer, or the model needs to apply some algorithm to summarize the too distant parts of conversation.

The shortcomings of making a context window larger include higher computational cost and possibly diluting the focus on local context, while making it smaller can cause a model to miss an important long-range dependency. Balancing them are a matter of experimental and domain-specific considerations.

A model may be pre-trained either to predict how the segment continues, or what is missing in the segment, given a segment from its training dataset.^[REDACTED] It can be either

- **autoregressive** (i.e. predicting how the segment continues, the way GPTs do it): for example given a segment "I like to eat", the model predicts "ice cream", or "honey".
- **masking** (i.e. filling in the parts missing from the segment, the way BERT^[REDACTED] does it): for example, given a segment "I like to eat [] [] cream", the model predicts that "eat" and "ice" are missing.

Models may be trained on auxiliary tasks which test their understanding of the data distribution, such as Next Sentence Prediction (NSP), in which pairs of sentences are presented and the model must predict whether they appear consecutively in the training corpus.^[REDACTED] During training, **regularization loss is also used to stabilize training. However regularization loss is usually not used during **testing** and evaluation.**

Training cost [\[edit\]](#)

Advances in software and hardware have reduced the cost substantially since 2020, such that in 2023 training of a 12-billion-parameter LLM computational cost is 7.20k A100-GPU-hours, while in 2020 the cost of training a 1.5-billion-parameter LLM was two orders of magnitude smaller than the state of the art in 2020 was between \$80 and \$140 million.^[REDACTED] Since 2020, large sums were invested in increasingly large models. For example, training of the GPT-2 (i.e. a 1.5-billion-parameters model) in 2019 cost \$50,000, while training of the PaLM (i.e. a 540-billion-parameters model) in 2022 cost \$8 million, and Megatron-Turing NLO G30B (in 2022) cost around \$11 million.^[REDACTED]

For Transformer-based LLM, training cost is much higher than inference cost. It costs 6 FLOPs per parameter to train on one token, whereas it costs 1 to 2 FLOPs per parameter to infer on one token.^[REDACTED]

MOUSE OF EXPERTS [\[edit\]](#)

MoE article: *Mixture of experts*

The original LLM was too expensive to train and use directly. For such models, **mixture of experts** (MoE) can be applied, a line of research pursued by Google researchers since 2017 to train models reaching up to 1 trillion parameters.^[REDACTED]

Prompt engineering, attention mechanism, and context window [\[edit\]](#)

See also: *Prompt engineering* and *Attention (machine learning)*

Most results previously achievable only by (costly) fine-tuning, can be achieved through **prompt engineering**, although limited to the scope of a single conversation (more precisely, limited to the scope of a context window)^[REDACTED]

Length of a conversation that the model can take into account when generating its next answer is limited by the size of a context window, as well. If the length of a conversation, for example with *Chat-GPT*, is longer than its context window, only the parts inside the context window are taken into account when generating the next answer, or the model needs to apply some algorithm to summarize the too distant parts of conversation.

The shortcomings of making a context window larger include higher computational cost and possibly diluting the focus on local context, while making it smaller can cause a model to miss an important long-range dependency. Balancing them are a matter of experimental and domain-specific considerations.

A model may be pre-trained either to predict how the segment continues, or what is missing in the segment, given a segment from its training dataset.^[REDACTED] It can be either

- **autoregressive** (i.e. predicting how the segment continues, the way GPTs do it): for example given a segment "I like to eat", the model predicts "ice cream", or "honey".
- **masking** (i.e. filling in the parts missing from the segment, the way BERT^[REDACTED] does it): for example, given a segment "I like to eat [] [] cream", the model predicts that "eat" and "ice" are missing.

Models may be trained on auxiliary tasks which test their understanding of the data distribution, such as Next Sentence Prediction (NSP), in which pairs of sentences are presented and the model must predict whether they appear consecutively in the training corpus.^[REDACTED] During training, **regularization loss is also used to stabilize training. However regularization loss is usually not used during **testing** and evaluation.**

Training cost [\[edit\]](#)

Advances in software and hardware have reduced the cost substantially since 2020, such that in 2023 training of a 12-billion-parameter LLM computational cost is 7.20k A100-GPU-hours, while in 2020 the cost of training a 1.5-billion-parameter LLM was two orders of magnitude smaller than the state of the art in 2020 was between \$80 and \$140 million.^[REDACTED] Since 2020, large sums were invested in increasingly large models. For example, training of the GPT-2 (i.e. a 1.5-billion-parameters model) in 2019 cost \$50,000, while training of the PaLM (i.e. a 540-billion-parameters model) in 2022 cost \$8 million, and Megatron-Turing NLO G30B (in 2022) cost around \$11 million.^[REDACTED]

For Transformer-based LLM, training cost is much higher than inference cost. It costs 6 FLOPs per parameter to train on one token, whereas it costs 1 to 2 FLOPs per parameter to infer on one token.^[REDACTED]

MOUSE OF EXPERTS [\[edit\]](#)

MoE article: *Mixture of experts*

The original LLM was too expensive to train and use directly. For such models, **mixture of experts** (MoE) can be applied, a line of research pursued by Google researchers since 2017 to train models reaching up to 1 trillion parameters.^[REDACTED]

Prompt engineering, attention mechanism, and context window [\[edit\]](#)

See also: *Prompt engineering* and *Attention (machine learning)*

Most results previously achievable only by (costly) fine-tuning, can be achieved through **prompt engineering**, although limited to the scope of a single conversation (more precisely, limited to the scope of a context window)^[REDACTED]

Length of a conversation that the model can take into account when generating its next answer is limited by the size of a context window, as well. If the length of a conversation, for example with *Chat-GPT*, is longer than its context window, only the parts inside the context window are taken into account when generating the next answer, or the model needs to apply some algorithm to summarize the too distant parts of conversation.

The shortcomings of making a context window larger include higher computational cost and possibly diluting the focus on local context, while making it smaller can cause a model to miss an important long-range dependency. Balancing them are a matter of experimental and domain-specific considerations.

A model may be pre-trained either to predict how the segment continues, or what is missing in the segment, given a segment from its training dataset.^[REDACTED] It can be either

- **autoregressive** (i.e. predicting how the segment continues, the way GPTs do it): for example given a segment "I like to eat", the model predicts "ice cream", or "honey".
- **masking** (i.e. filling in the parts missing from the segment, the way BERT^[REDACTED] does it): for example, given a segment "I like to eat [] [] cream", the model predicts that "eat" and "ice" are missing.

Models may be trained on auxiliary tasks which test their understanding of the data distribution, such as Next Sentence Prediction (NSP), in which pairs of sentences are presented and the model must predict whether they appear consecutively in the training corpus.^[REDACTED] During training, **regularization loss is also used to stabilize training. However regularization loss is usually not used during **testing** and evaluation.**

Training cost [\[edit\]](#)

Advances in software and hardware have reduced the cost substantially since 2020, such that in 2023 training of a 12-billion-parameter LLM computational cost is 7.20k A100-GPU-hours, while in 2020 the cost of training a 1.5-billion-parameter LLM was two orders of magnitude smaller than the state of the art in 2020 was between \$80 and \$140 million.^[REDACTED] Since 2020, large sums were invested in increasingly large models. For example, training of the GPT-2 (i.e. a 1.5-billion-parameters model) in 2019 cost \$50,000, while training of the PaLM (i.e. a 540-billion-parameters model) in 2022 cost \$8 million, and Megatron-Turing NLO G30B (in 2022) cost around \$11 million.^[REDACTED]

For Transformer-based LLM, training cost is much higher than inference cost. It costs 6 FLOPs per parameter to train on one token, whereas it costs 1 to 2 FLOPs per parameter to infer on one token.^[REDACTED]

Mixture of experts [edit]

Main article: *Mixture of experts*

The largest LLM may be too expensive to train and use directly. For such models, *mixture of experts* (MoE) can be applied, a line of research pursued by Google researchers since 2017 to train models reaching up to 1 trillion parameters.^{[28][29][30]}

Prompt engineering, attention mechanism, and context window [edit]

See also: *Prompt engineering* and *Attention (machine learning)*

Most results previously achievable only by (costly) fine-tuning, can be achieved through *prompt engineering*, although limited to the scope of a single conversation (more precisely, limited to the scope of a context window).^[31]

In order to find out which tokens are relevant to each other within the scope of the context window, the attention mechanism calculates "soft" weights for each token, more precisely for its embedding, by using multiple attention heads, each with its own "relevance" for calculating its own soft weights. For example, the small (i.e. 117M parameter sized) GPT-2 model, has had twelve attention heads and a context window of only 1k token.^[32] In its medium version it has 345M parameters and contains 24 layers, each with 12 attention heads. For the training with gradient descent a batch size of 512 was utilized.^[33]

The largest models, such as Google's *Gemini 1.5*, presented in February 2024, can have a context window sized up to 1 million (context window of 10 million was also "successfully tested").^[34] Other models with large context windows includes Anthropic's Claude 2.1, with a context window of up to 200k tokens.^[35] Note that this maximum refers to the number of input tokens and that the maximum number of output tokens differs from the input and is often smaller. For example, the GPT-4 Turbo model has a maximum output of 4096 tokens.^[36]

Length of a conversation that the model can take into account when generating its next answer is limited by the size of a context window, as well. If the length of a conversation, for example with Chat-GPT, is longer than its context window, only the parts inside the context window are taken into account when generating the next answer, or the model needs to apply some algorithm to summarize the too distant parts of conversation.

The shortcomings of making a context window larger include higher computational cost and possibly limiting the focus on local context, while making it smaller can cause a model to miss an important long-range dependency. Balancing them are a matter of experimentation and domain-specific considerations.

A model may be pre-trained either to predict how the segment continues, or what is missing in the segment, given a segment from its training dataset.^[37] It can be either

- autoregressive (i.e. predicting how the segment continues, the way GPTs do it); for example

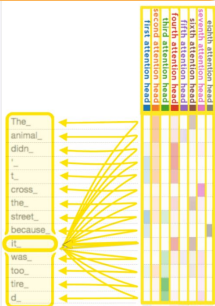
- "masked" (i.e. filling in the parts missing from the segment, the way BERT^[38] does it); for example, given a segment "I like to [] [] cream" the model predicts that "eat" and "ice" are missing.

Models may be trained on auxiliary tasks which test their understanding of the data distribution, such as Next Sentence Prediction (NSP), in which pairs of sentences are presented and the model must predict whether they appear consecutively in the training corpus.^[39] During training, regularization loss is also used to stabilize training. However regularization loss is usually not used during testing and evaluation.

Training cost [edit]

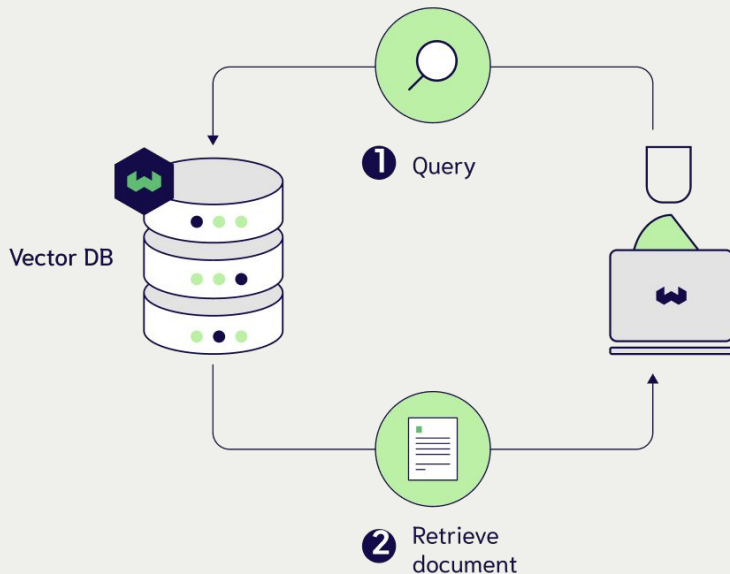
Advances in software and hardware have reduced the cost substantially since 2020, such that in 2023 training of a 12-billion-parameter LLM computational cost is 72,300 A100-GPU-hours, while in 2020 the cost of training a 1.5-billion-parameter LLM (which was two orders of magnitude smaller than the state of the art in 2020) was between \$80 thousand and \$1.6 million.^{[39][40][41]} Since 2020, large sums were invested in increasingly large models. For example, training of the GPT-2 (i.e. a 1.5-billion-parameters model) in 2019 cost \$50,000, while training of the PaLM (i.e. a 540-billion-parameters model) in 2022 cost \$8 million, and megatron-turing NLG 530B (in 2022) cost around \$11 million.^[42]

For Transformer-based LLM, training cost is much higher than inference cost. It costs 6 FLOPs per parameter to train on one token, whereas it costs 1 to 2 FLOPs per parameter to infer on one token.^[43]



When each head calculates, according to its own criteria, how much other tokens are relevant for the "it_" token, note that the second attention head, represented by the second column, is focusing most on the first two rows, i.e. the tokens "The" and "animal", while the third column is focusing most on the bottom two rows, i.e. on "tired", which has been tokenized into two tokens.^[34]

Chunking & retrieval





Mixture of experts [edit]

Main article: *Mixture of experts*

The largest LLM may be too expensive to train and use directly. For such models, *mixture of experts* (MoE) can be applied, a line of research pursued by Google researchers since 2017 to train models reaching up to 1 trillion parameters.^{[28][29][30]}

Prompt engineering, attention mechanism, and context window [edit]

See also: *Prompt engineering* and *Attention (machine learning)*

Most results previously achievable only by (costly) fine-tuning, can be achieved through *prompt engineering*, although limited to the scope of a single conversation (more precisely, limited to the scope of a context window).^[31]

In order to find out which tokens are relevant to each other within the scope of the context window, the attention mechanism calculates "soft" weights for each token, more precisely for its embedding, by using multiple attention heads, each with its own "relevance" for calculating its own soft weights. For example, the small (i.e. 117M parameter sized) GPT-2 model, has had twelve attention heads and a context window of only 1k token.^[32] In its medium version it has 345M parameters and contains 24 layers, each with 1k attention heads. For the training with gradient descent a batch size of 512 was utilized.^[33]

The largest models, such as Google's *Gemini 1.5*, presented in February 2024, can have a context window sized up to 1 million (context window of 10 million was also "successfully tested").^[34] Other models with large context windows includes Anthropic's Claude 2.1, with a context window of up to 200k tokens.^[35] Note that this maximum refers to the number of input tokens and that the maximum number of output tokens differs from the input and is often smaller. For example, the GPT-4 Turbo model has a maximum output of 4096 tokens.^[36]

Length of a conversation that the model can take into account when generating its next answer is limited by the size of a context window, as well. If the length of a conversation, for example with Chat-GPT, is longer than its context window, only the parts inside the context window are taken into account when generating the next answer, or the model needs to apply some algorithm to summarize the too distant parts of conversation.

The shortcomings of making a context window larger include higher computational cost and possibly *distilling* the focus on local context, while making it smaller can cause a model to miss an important long-range dependency. Balancing them are a matter of experimentation and domain-specific considerations.

A model may be pre-trained either to predict how the segment continues, or what is missing in the segment, given a segment from its training dataset.^[37] It can be either

- *autoregressive* (i.e. predicting how the segment continues, the way *GPTs* do it); for example

- *"masked"* (i.e. filling in the parts missing from the segment, the way "BERT"^[38] does it); for example, given a segment "I like to [] [] cream" the model predicts that "eat" and "ice" are missing.

Models may be trained on *auxiliary tasks* which test their understanding of the data distribution, such as Next Sentence Prediction (NSP), in which pairs of sentences are presented and the model must predict whether they appear consecutively in the training corpus.^[39] During training, regularization loss is also used to stabilize training. However regularization loss is usually not used during testing and evaluation.

Training cost [edit]

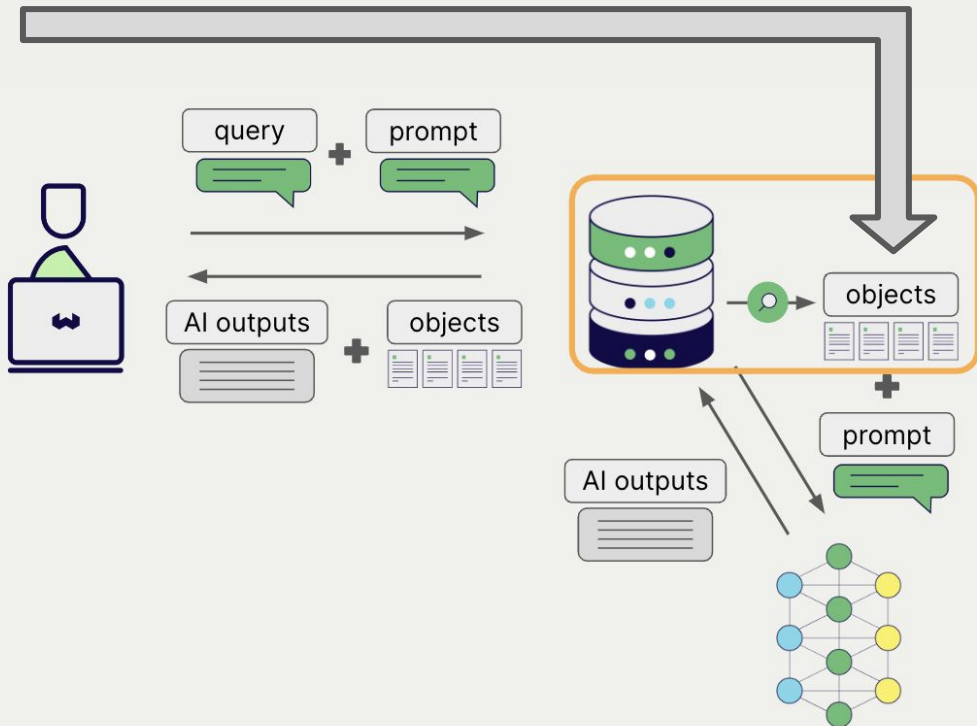
Advances in software and hardware have reduced the cost substantially since 2020, such that in 2023 training of a 12-billion-parameter LLM computational cost is 72,300 A100-GPU-hours, while in 2020 the cost of training a 1.5-billion-parameter LLM (which was two orders of magnitude smaller than the state of the art in 2020) was between \$80 thousand and \$1.6 million.^{[38][40][41]} Since 2020, large sums were invested in increasingly large models. For example, training of the GPT-2 (i.e. a 1.5-billion-parameters model) in 2019 cost \$50,000, while training of the PaLM (i.e. a 540-billion-parameters model) in 2022 cost \$8 million, and megatron-turing NLG 530B (in 2021) cost around \$11 million.^[42]

For Transformer-based LLM, training cost is much higher than inference cost. It costs 6 FLOPs per parameter to train on one token, whereas it costs 1 to 2 FLOPs per parameter to infer on one token.^[43]



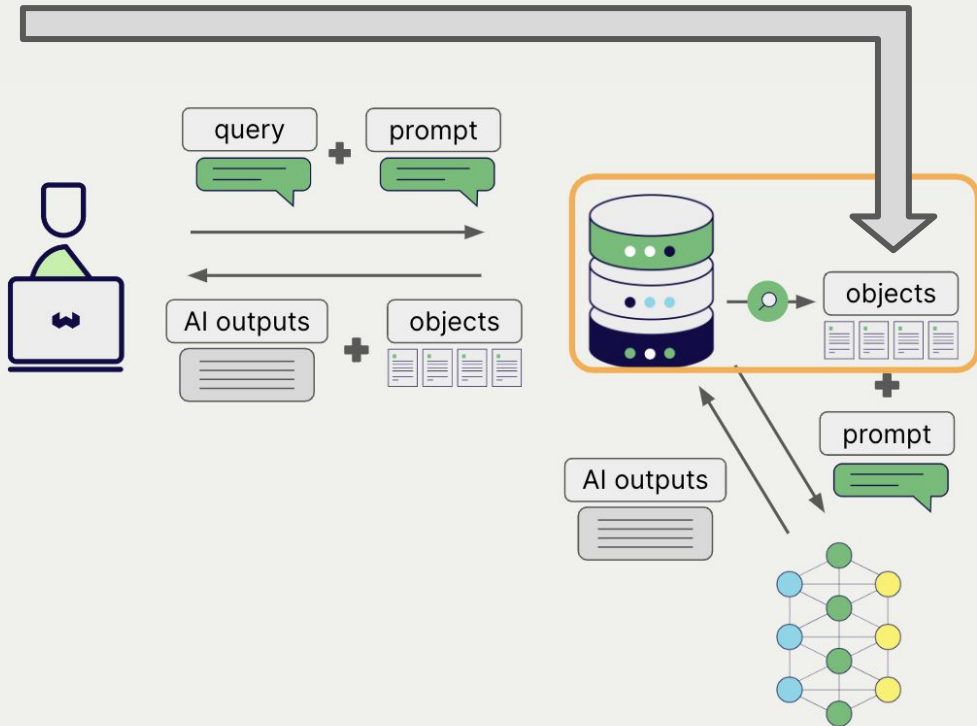
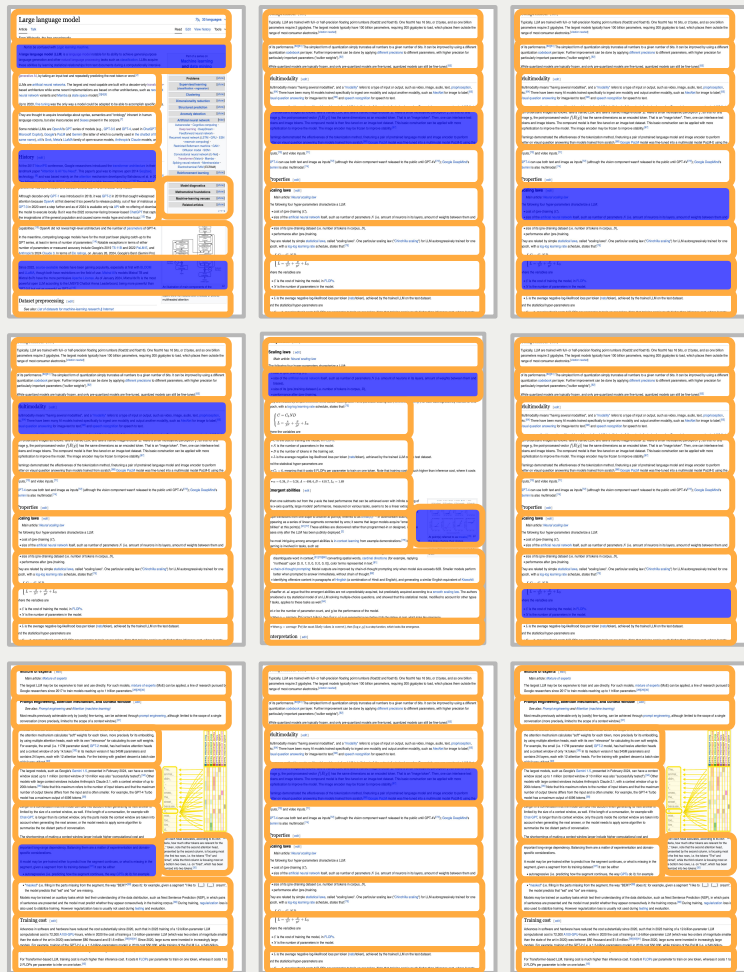
When each head calculates, according to its own criteria, how much other tokens are relevant for the "I_" token, note that the second attention head, represented by the second column, is focusing most on the first two rows, i.e. the tokens "The" and "animal", while the third column is focusing most on the bottom two rows, i.e. on "fired", which has been tokenized into two tokens.^[34]

Chunking & RAG





Chunking & RAG





Advanced IT software and FableNews Kera reduced the cost substantially since 2019, with P&M in 2022 training of a 174-billion-parameter LLM computational cost to 72,300 A100 GPU-hours, while in 2020 the cost of training a 1.5-billion-parameter LLM (which was two orders of magnitude smaller) was 1,000 A100 GPU-hours. For example, training of the GPT-4 (i.e. a 1.5-trillion-parameter model) in 2019 cost \$500 million, while training of the PaLM (i.e. a 540-billion-parameter model) in 2022 cost \$1 million, and Microsoft's Syntex NLP-500B (in 2022) cost \$1 million (see Table 1).

Chunking

Trade-offs

- Local precision vs overall context
- **Larger** chunks: more context / more noise
- **Smaller** chunks: precision / less context

Chunking

Start simple

- 100-150 word chunks
- Semantic boundaries
- Some overlap

Chunking

Iterate

- Explore sizing
- Get neighbours
 - "small2big"
- Mix strategies
- Don't overcomplicate it

```
chunks = source_text.split("\n==") # Split the text by AsciiDoc marker

new_chunks = list()
chunk_buffer = ""
min_length = 25

for chunk in chunks:
    new_buffer = chunk_buffer + chunk # Create new buffer
    new_buffer_words = new_buffer.split(" ") # Split into words
    if len(new_buffer_words) < min_length: # Check whether buffer length too small
        chunk_buffer = new_buffer # Carry over to the next chunk
    else:
        new_chunks.append(new_buffer) # Add to chunks
        chunk_buffer = ""

if len(chunk_buffer) > 0:
    new_chunks.append(chunk_buffer) # Add last chunk, if necessary
```




Key topics

Vectorizer model choice

Horses for courses

Chunking

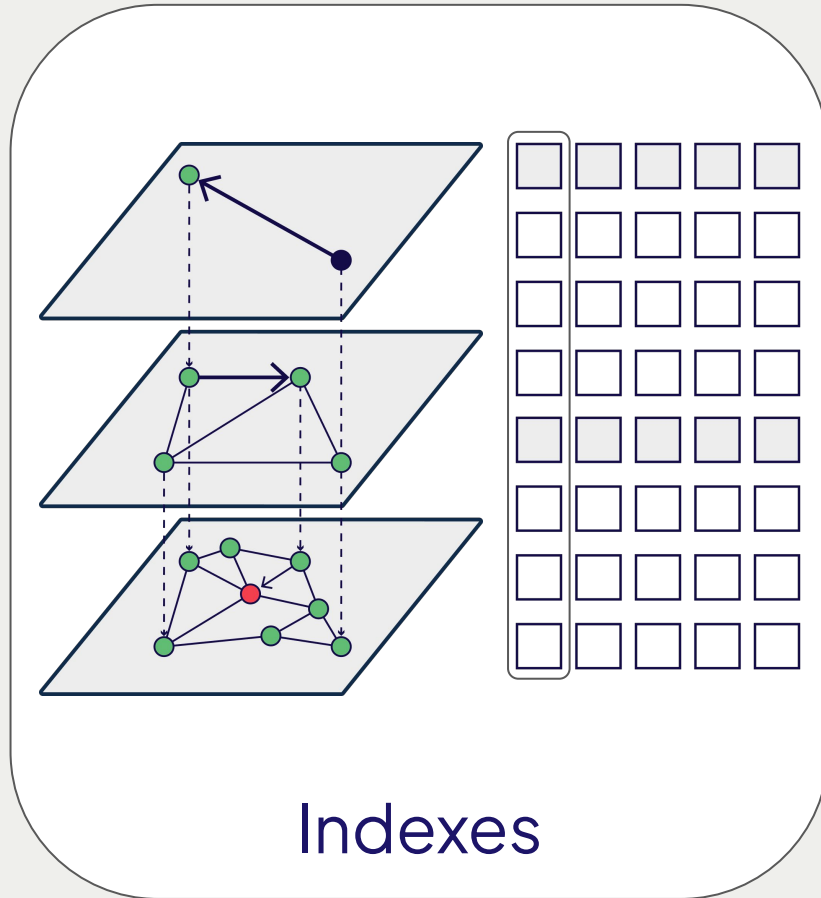
Start with end in mind

Quantization

Resource management

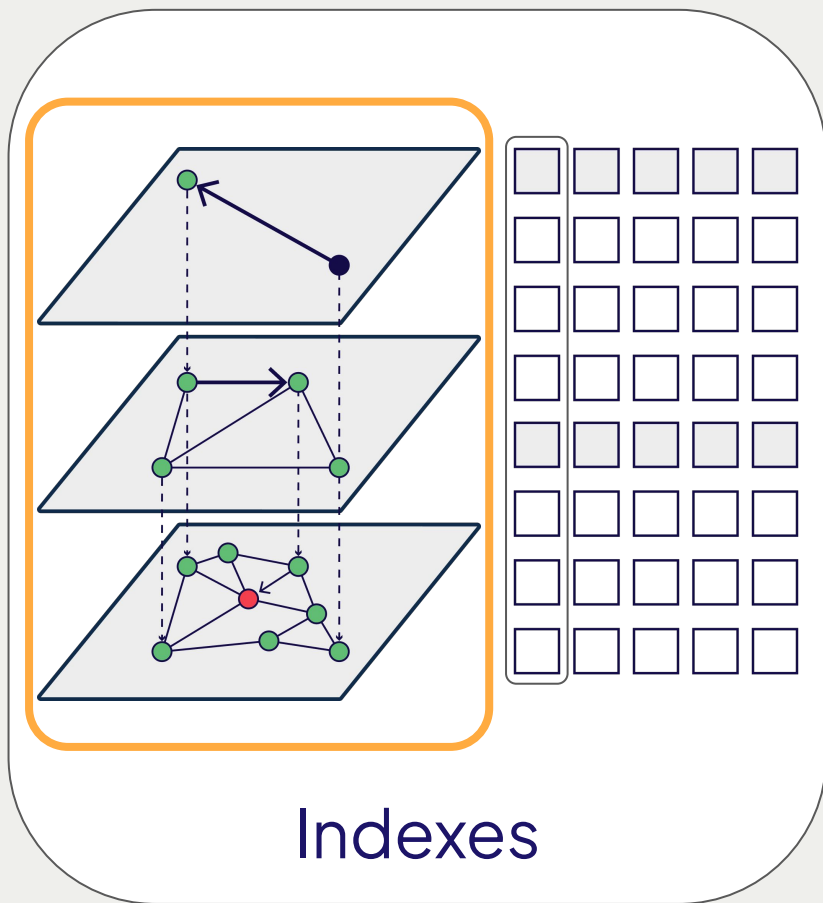
Vector construction

Define “similarity”



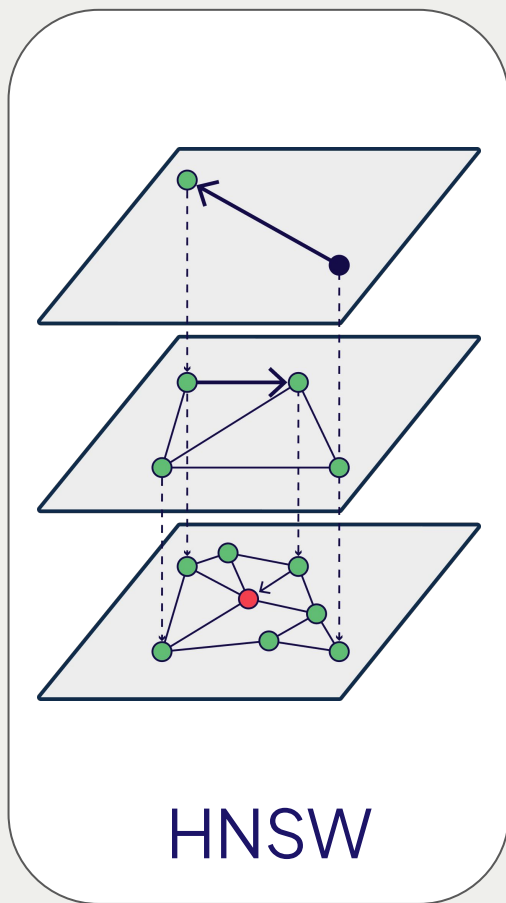
Indexes:

- Catalogue of data
 - Can speed up search
- Exist in RDBMS



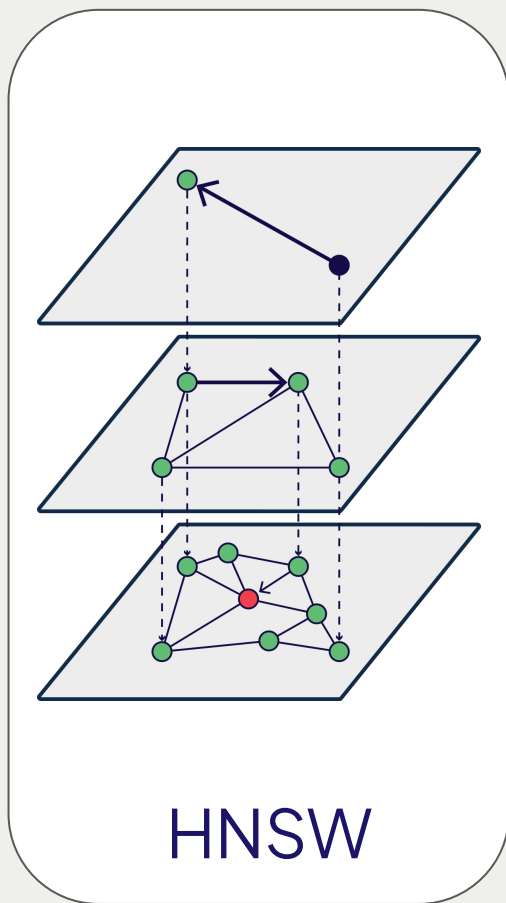
Graph-based indexes:

- Common (e.g. HNSW)
- Scalable
- Typical limitation:
 - Memory footprint



HNSW in focus

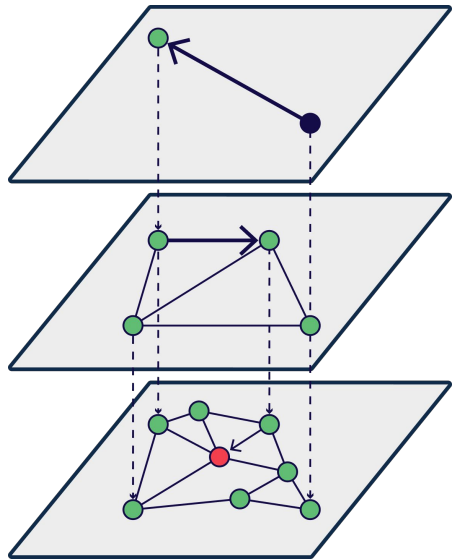
- In-memory data structure
- Typical limitation: memory footprint
 - Vectors ($\sim 4\text{B}/\text{dim}$)
 - Connections ($\sim 10\text{B}/\text{ea}$)
 - @10M objects
 - $\sim 65\text{GB}$!
 - @100M $\rightarrow 650\text{GB}$, etc...



Quantization

- Compression
- Reduce precision from floats
 - Binary / scalar / locally adaptive
- Reduce number of dimensions
 - Product quantization

Binary Quantization

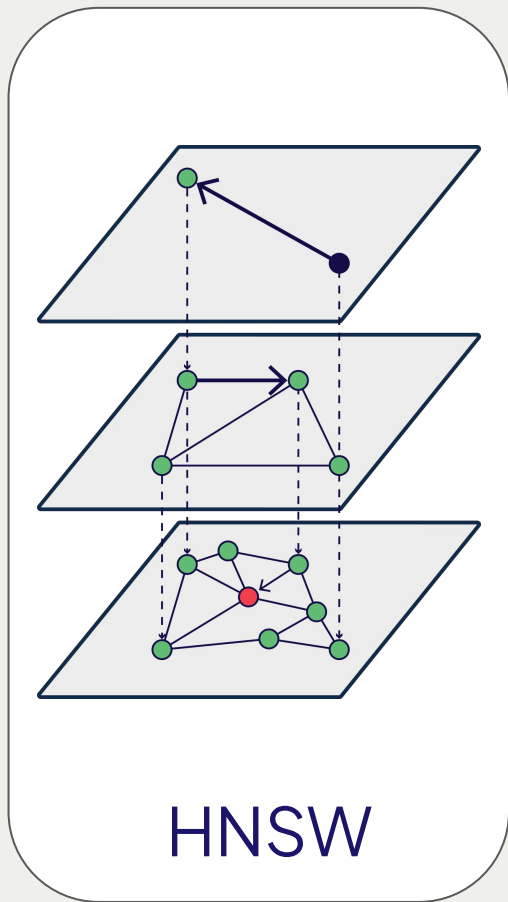


HNSW

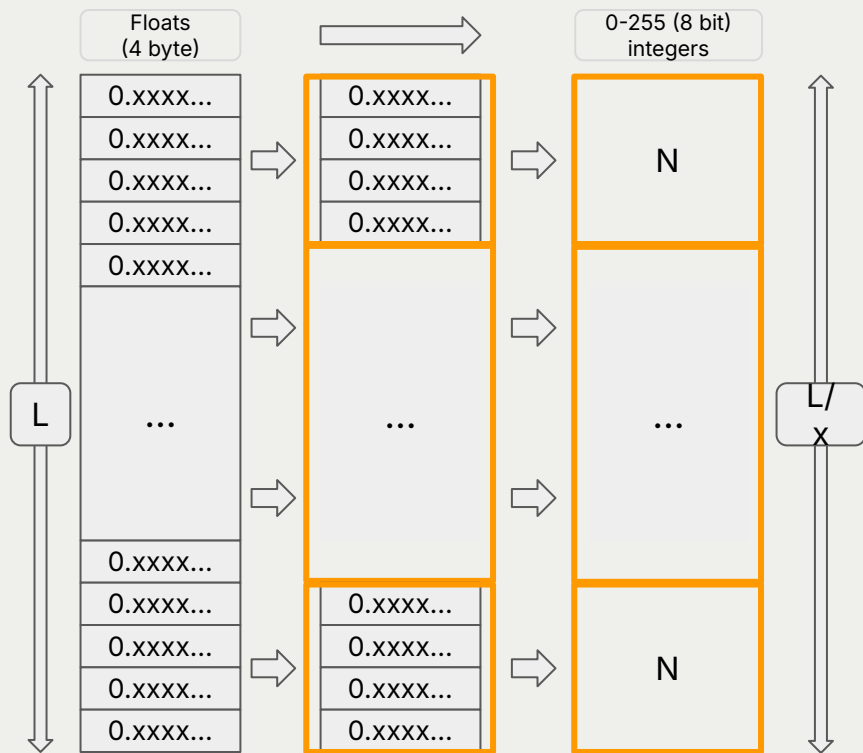
$[-0.1324..., -0.9253...,$
 $0.2389...,$
 $\dots$
 $0.3249..., 0.2390..., -0.4823...]$

↓

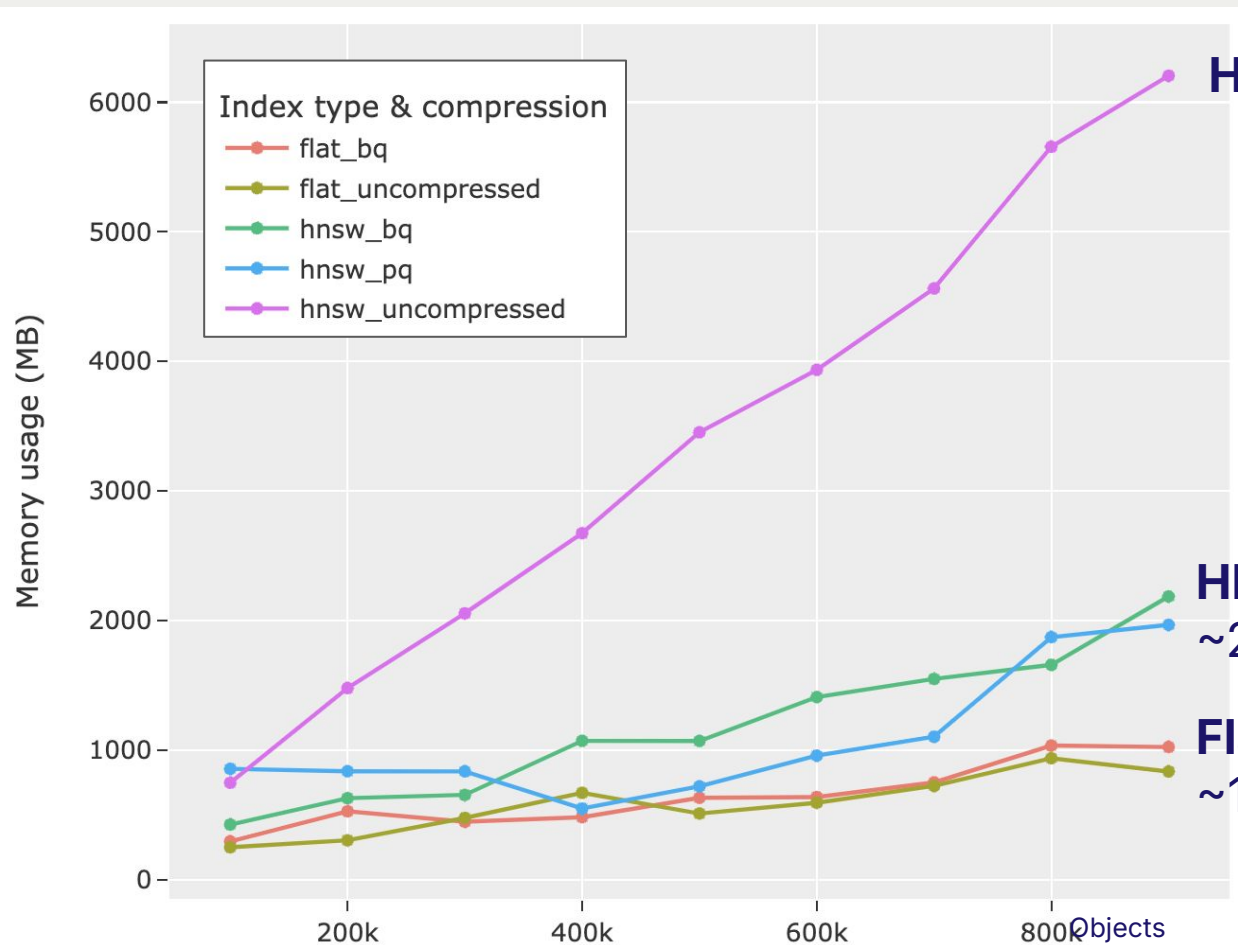
$[0, 0, 1, 0, 1, 1,$
 $\dots$
 $0, 1, 0, 1, 1, 0]$



Product Quantization



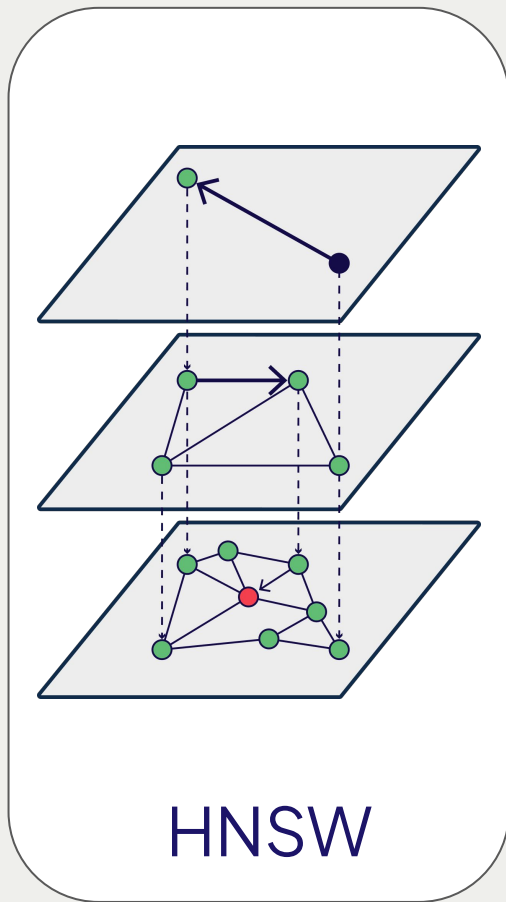
Example: Index type & quantization vs memory footprint



HNSW: ~6GB

HNSW + BQ/PQ:
~2GB

Flat (+BQ):
~1GB



Quantization

- If data size uncertain - start with quantization
- Use over-fetching & rescoring
- Use default values



Key topics

Vectorizer model choice

Horses for courses

Chunking

Start with end in mind

Quantization

Resource management

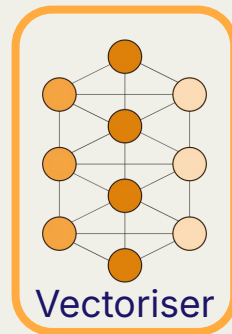
Vector construction

Define “similarity”

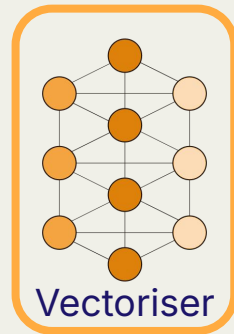
Vector construction

Which text fields to use?

- From: [title, abstract, chunk, author, ...]
- More isn't better
- Context vs precision



Vector construction



How many vectors per object?

- Use different fields?
 - Movie DB: Title / Summary / Title + summary
- Use multiple modalities?
 - Movie DB: Text vector(s) + Image vector
- Depends on the goal & usage

Vector DB: Data ingestion strategies

- Define clear goals
- Resource requirements
- Start small & simple
 - Iterate with e2e experiments
- Keep an open mind

Try



<https://hubs.ly/Q02tMNTM0>